

Object-Oriented Languages

Chapter 14

Object-Oriented Languages

- **Class-based, object-oriented (OO)** language
 - All (or most) values are objects
 - An object is an instance of a class
 - Objects encapsulate **state** (fields) and **behavior** (methods)
- Some important features of OO languages
 - Inheritance
 - Encapsulation
 - Polymorphism

Outline

- Classes
- Single Inheritance of Data Fields
- Multiple Inheritance
- Testing Class Membership
- Private Fields and Methods

Outline

- **Classes**
- Single Inheritance of Data Fields
- Multiple Inheritance
- Testing Class Membership
- Private Fields and Methods

Object-Tiger

- Extend the Tiger language with new declaration syntax to create classes:

dec → *classdec*

classdec → **class** *class-id* **extends** *class-id* { {*classfield* } }

classfield → *vardec*

classfield → *method*

method → **method** *id*(*tyfields*) = *exp*

method → **method** *id*(*tyfields*) : *type-id* = *exp*

Object-Tiger

- `class B extends A { ... }`
 - declares a new class `B` that extends class `A`
 - must be **in the scope** of the let-expression **that declares `A`**
 - All the fields and methods of `A` implicitly belong to `B`.
 - Some of the `A` methods may be overridden (have new declarations) in `B`. The parameter and result types must be identical.
 - But the fields may not be overridden.
- There is a predefined class identifier `Object` with no fields or methods.
- Each method within class `B` has an implicit formal parameter `self` of type `B`.
 - `self`: not a reserved word, just an identifier automatically bound in each method

Object Tiger

- New expression syntax to create objects and invoke methods
 - `new B`
 - `b.x`
 - `b.f(x, y)`: the value `b` for the implicit `self` parameter of `f`

exp → **new** *class-id*
→ *lvalue* . *id*()
→ *lvalue* . *id*(*exp*{, *exp*})

An Object-Oriented Program

```
let start := 10
class Vehicle extends Object {
  var position := start
  method move(int x) = (position := position + x)
}
class Truck extends Vehicle {
  method move(int x) =
    if x <= 55
    then position := position + x
}
class Car extends Vehicle {
  var passengers := 0
  method await(v: Vehicle) =
    if (v.position < position)
    then v.move(position - v.position)
    else self.move(10)
}
...
in
...
end
```

What are the
variables in scope
on entry to `await`?

An Object-Oriented Program (Cont)

```
let start := 10
...
class Car extends Vehicle {
  var passengers := 0
  method await(v: Vehicle) =
    if(v.position < position)
    then v.move(position - v.position)
    else self.move(10)
}
var t := new Truck
var c := new Car
var v : Vehicle := c
in
  c.passengers := 2;
  c.move(60);
  v.move(70);
  c.await(t)
end
```

Generate Code to Fetch Fields

- `v.position`
 - `v` belongs to class `Vehicle`
 - To evaluate it, the compiler must generate code to fetch the field `position` from the object (record) that `v` points to.
- How?
- Simple idea:
 - Get the ***class descriptor*** of `Vehicle` from the environment entry of variable `v`
 - Get the offset of `position` from this descriptor
- But at run time:
 - `v` could contain a pointer to a `Car` or `Truck`.
 - Where will the `position` field be?

Outline

- Classes
- **Single Inheritance of Data Fields**
- Multiple Inheritance
- Testing Class Membership
- Private Fields and Methods

Single Inheritance

- **Single-inheritance** languages:
 - Each class extends just one parent class
- For **single-inheritance** languages, how to generate code to fetch fields?

Fields

- **Prefixing**

- Where **B** extends **A**, those fields of **B** that are inherited from **A** are laid out in a **B** record at the beginning, in the same order they appear in **A** records.
- Fields of **B** not inherited from **A** are placed afterward.

```
class A extends Object { var a := 0 }  
class B extends A { var b := 0  
                  var c := 0 }  
class C extends A { var d := 0 }  
class D extends B { var e := 0 }
```

A
a

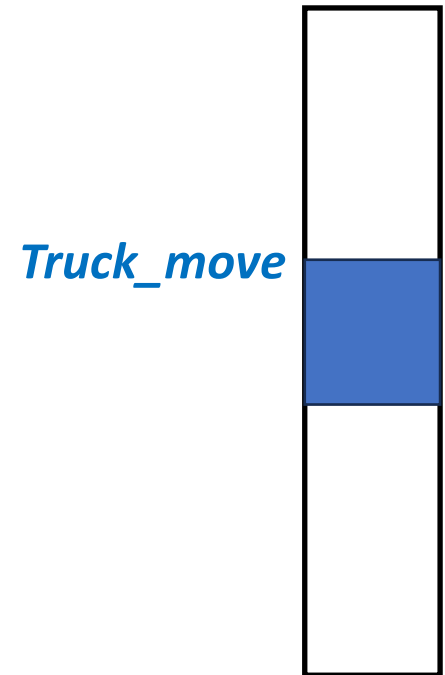
B
a
b
c

C
a
d

D
a
b
c
e

Methods

- A **method** instance is compiled much like a **function**
 - It turns into machine code that resides at a **particular address** in the instruction space
- For example, the method instance *Truck_move* has an entry point at machine-code label *Truck_move*.
- Each class descriptor contains a **pointer to its parent class**, and also a **list of method instances**.



Instruction Space

Static Methods

- Some object-oriented languages allow some methods to be declared *static*.
- To compile a method call of the form `c.f()`, the compiler:
 - finds the class of `c`, suppose it is `class C`
 - searches in `class C` for a method `f`, suppose none is found
 - searches the parent class of `C`, suppose it is `class B`, and so on
 - Suppose in some ancestor `class A` it finds a `static method f`, then it can compile a function call to label `A_f`

`c.f()`  `A_f`

Dynamic Methods

- What if method `f` in `A` is a dynamic method, can we convert `c.f()` to `A_f()`?
 - `f` may be overridden in some class `D` which is a subclass of `C`.
 - There is no way to tell at compile time if `c` is pointing to an object of class `D` (`D_f` should be called) or class `C` (`A_f` should be called)
- How to solve this problem?

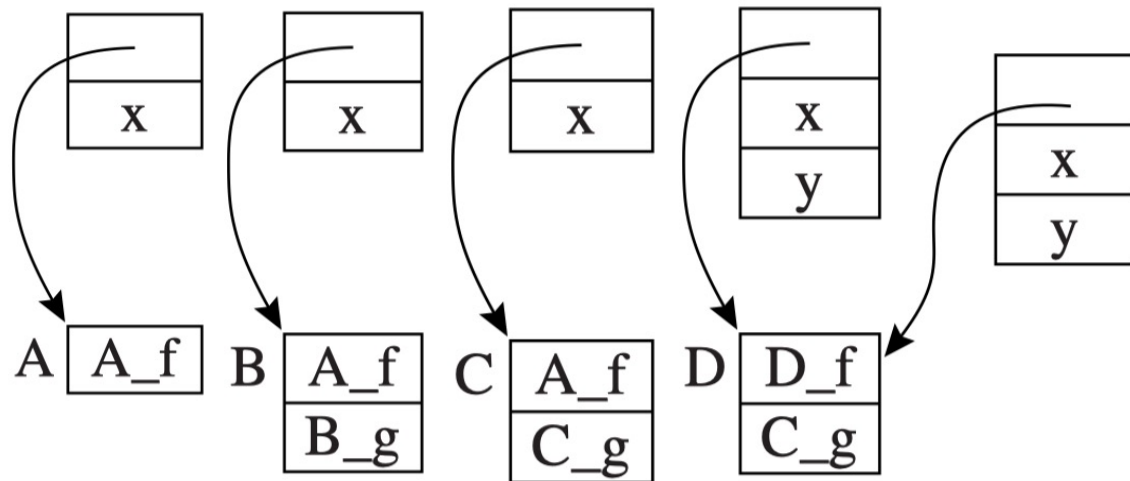
```
Class A extends Object {  
    var x := 0  
    method f() }  
Class B extends A {  
    method g() }  
Class C extends B {  
    method g() }  
Class D extends C {  
    var y := 0  
    method f() }
```

`c.f()`

Dynamic Methods

- The class descriptor must contain a vector (*dispatch vector*, *virtual table*, *vtable*) with a method instance for each (nonstatic) method name.
 - Prefixing:** when class **B** inherits from **A**, the method table starts with entries for all method names known to **A**, and then continues with new methods declared by **B**.

```
Class A extends Object {  
  var x := 0  
  method f() }  
Class B extends A {  
  method g() }  
Class C extends B {  
  method g() }  
Class D extends C {  
  var y := 0  
  method f() }
```



Dynamic Methods

- To execute `c.f()`, where `f` is a dynamic method, the compiled code must execute these instructions:
 1. Fetch the class descriptor `d` at offset 0 from object `c`.
 2. Fetch the method-instance pointer `p` from the (constant) `f` offset of `d`.
 3. Jumps to address `p`, saving return address (that is, call `p`).

Outline

- Classes
- Single Inheritance of Data Fields
- **Multiple Inheritance**
- Testing Class Membership
- Private Fields and Methods

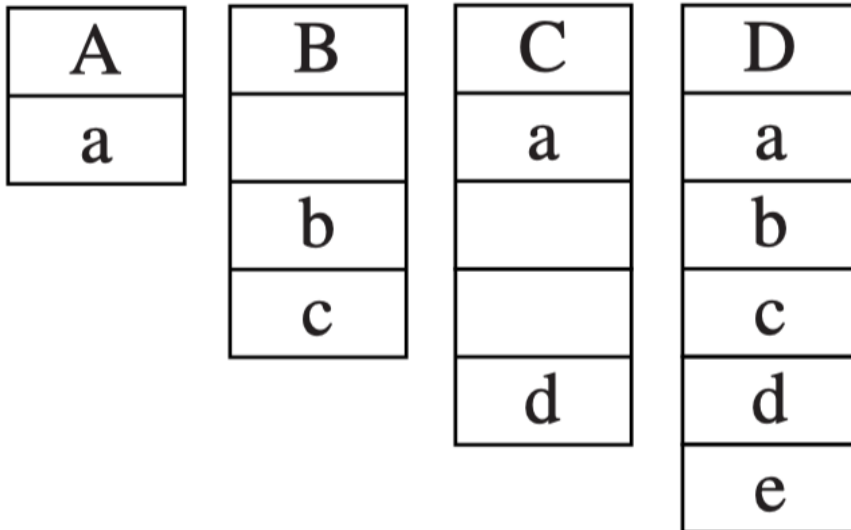
Multiple Inheritance

- In languages that permit a class **D** to extend several parent classes **A**, **B** and **C**, finding field offsets and method instances is more difficult.
 - It is impossible to put all the **A** fields at the beginning of **D** *and* to put all the **B** fields at the beginning of **D**.
- How to solve this problem?

Global Graph Coloring - Fields

- One Solution to this problem is to statically analyze all classes at once (using graph-coloring algorithm), finding **some offset for each field name that can be used in every record** containing that field.

```
class A extends Object {var a := 0 }  
class B extends Object {var b:=0  
                        var c:= 0 }  
class C extends A {var d:=0 }  
class D extends A,B,C { var e:=0 }
```



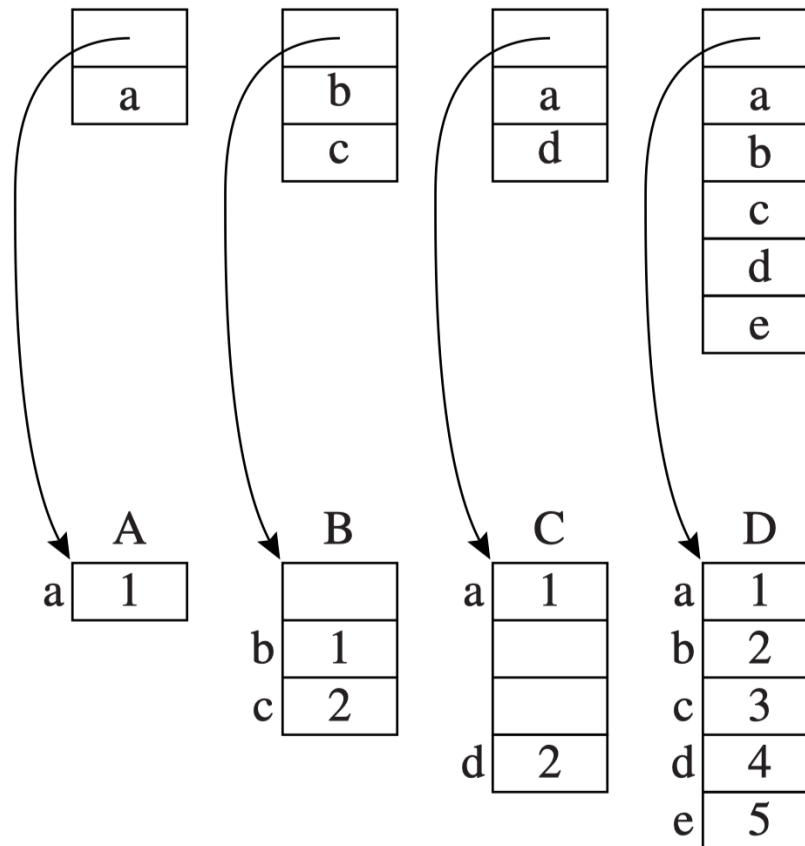
Graph-coloring

- node**: a distinct field name
- edge**: two fields coexist in a class
- colors**: offsets (0, 1, 2 ...)

Global Graph Coloring - Fields

- The problem with this approach is that it leaves **empty slots** in the middle of objects.
- Solution: Pack the fields of each object and have the class descriptor tell where each field is.

How to access
the field **c** of the
second object?



Global Graph Coloring - Fields

- In this scheme, class descriptors have empty slots, but the objects do not.
 - Acceptable: $\#objects \gg \#class\ descriptors$
- But each data fetch (or store) requires three instructions instead of one:
 1. Fetch the descriptor-pointer from the object.
 2. Fetch the field-offset value from the descriptor.
 3. Fetch (or store) the data at the appropriate offset in the object.
- Compare: for single-inheritance language, we can know the offset of each field at compilation time (using prefixing).

Global Graph Coloring – Method Lookup

- How to find method instances in a multiple-inheritance language?
- The global graph-coloring approach works well:
 - The method names can be mixed with the field names to form nodes of a large interference graph.
 - Descriptor entries for fields give **locations within the objects**
 - Descriptor entries for methods give **machine-code addresses of method instances**.

Global Graph Coloring - Problem

Problems with dynamic linking:

- The global coloring can be done only at **link-time**.
- However, many object-oriented systems have the capability to load new classes into a running system.
- Link-time graph coloring poses many problems for a system that **allows dynamic incremental linking**.

Hashing

- Put a hash table in each class descriptor, mapping field names to offsets and method names to method instances
- This works well with separate compilation and dynamic linking
- **Ftab**: *field-offset* table, contains field-offsets and method instances
- **Ktab**: *key* table, contains field-name pointers (for purposes of collision detection)
- If the class has a field **x**, then field-offset-table slot number **hash_x** contains the offset for **x**, and key-table slot number **hash_x** contains the pointer **ptr_x**.

Hashing

- To fetch a field x of object c , the compiler generates code to:
 1. Fetch the class descriptor d at offset 0 from object c .
 2. Fetch the field-name f from the address offset $d + Ktab + hash_x$.
 3. Test whether $f = ptr_x$; if so (no collision)
 4. Fetch the field offset k from $d + Ftab + hash_x$.
 5. Fetch the contents of the field from $c + k$.
- A similar algorithm works for dynamic method-instance lookup.
- Any hash-table collision-resolution technique can be used.

Outline

- Classes
- Single Inheritance of Data Fields
- Multiple Inheritance
- **Testing Class Membership**
- Private Fields and Methods

Testing Class Membership

- Some object-oriented languages allow the program to test membership of an object in a class at run time
 - For example

	Modula-3	Java
Test whether object x belongs class C , or to any subclass of C .	<code>ISTYPE (x, C)</code>	<code>x instanceof C</code>
Given a variable x of class C , where x actually points to an object of class D that extends C , yield an expression whose compile-time type is class D .	<code>NARROW (x, D)</code>	<code>(D) x</code>

TABLE 14.6 Facilities for type testing and safe casting

Testing Class Membership

- Assuming there is no multiple inheritance, a simple way to implement `x instanceof C` is to generate code that performs the following loop at run time:

```
t1 ← x.descriptor  
L1: if t1 = C goto true  
    t1 ← t1.super  
    if t1 = nil goto false  
    goto L1
```

Can be slow

where `t1.super` is the superclass (parent class) of class `t1`.

Testing Class Membership - Display

- A faster approach: using a *display* of parent classes
- Assume that the *class-nesting depth* is limited to some constant, such as 20. Reserve a 20-word block in each class descriptor.
- Suppose the nesting depth of *class D* is *j* (which can be known at compile time)
 - In *D*'s descriptor,
 - *display[j] = D*
 - *display[j-1] = D.super*
 - *display[j-2] = D.super.super,*
 - ...,
 - *display[0] = Object*
 - *display[k] = nil*, where $k > j$

Testing Class Membership - Display

- Now, if x is an instance of D , or of any subclass of D , in x 's class descriptor, $\text{display}[j] = D$. Otherwise it will not.
- So $x \text{ instanceof } D$ requires:
 1. Fetch the class descriptor d at offset 0 from object x .
 2. Fetch the j th class-pointer slot from d .
 3. Compare with the class descriptor D .

Type Coercions

- Given a variable c of type C
 - Treat c as any supertype of C : legal and safe
 - e.g., $\text{var } b : B := c$, where C extends B
- The reverse is not true.
 - The assignment $c \leftarrow b$ is safe only if b is really (at run time) an instance of C , which is not always the case.

```
b ← new B  
c ← b  
c.some_field_of_C_but_not_B
```

Unpredictable
behavior

Type Coercions

- Some object-oriented languages (such as **Modula-3** and **Java**) accompany any coercion from a superclass to a subclass with **a run-time type-check**:
 - raises an exception unless the run-time value is really an instance of the subclass (e.g., unless **b instanceof C**)
 - Example

```
Modula-3:  
IF ISTYPE(b,C)  
  THEN f(NARROW(b,C))  
ELSE ...
```

```
Java:  
if (b instanceof C)  
  f((C)b)  
else ...
```

C++ has a static cast mechanism without run-time checking
=> **unsafe**

Typecase

- **Module-3** has a **typecase** facility that makes the **test-then-narrow idiom** more beautiful and efficient.

```
TYPECASE expr
OF C1 (v1) => S1
|  C2 (v2) => S2
.
.
.
|  Cn (vn) => Sn
ELSE S0
END
```

- If more than one of the **Ci** match (e.g., one is a superclass of another), then **only the first matching clause** is taken.
- If none of the **Ci** match, then the **ELSE** clause is take.

Typecase

- **Typecase** can be converted straightforwardly to a chain of **else-ifs**, with each **if** doing:
 - an instance test
 - a narrowing
 - a local variable declaration

```
TYPECASE expr
OF C1 (v1) => S1
|  C2 (v2) => S2
.
.
.
|  Cn (vn) => Sn
ELSE S0
END
```

Outline

- Classes
- Single Inheritance of Data Fields
- Multiple Inheritance
- Testing Class Membership
- **Private Fields and Methods**

Private Fields and Methods

- True object-oriented languages can protect fields of objects from direct manipulation by other objects' methods.
- A **private** field is one that cannot be fetched or updated from any function or method declared outside the object
- A **private** method is one that cannot be called from outside the object.
- Privacy is enforced by the type-checking phase of the compiler.
- How to check?
- In the symbol table of class **C**, along with each field offset and method offset, is a **boolean flag** indicating whether the field is private.

Privacy and Protection

There are many varieties of privacy and protection. Different languages allow:

- Fields and methods which are accessible only to **the class that declares them**.
- Fields and methods accessible to **the declaring class**, and to **any subclasses of that class**.
- Fields and methods accessible only **within the same module** (package, namespace) as the declaring class.
- Fields that are **read-only from outside the declaring class**, but writable by methods of the class.

In general, these are statically enforced by **compile-time type-checking**

Summary

- Classes
- Single Inheritance of Data Fields
- Multiple Inheritance
- Testing Class Membership
- Private Fields and Methods